

DOINGG@UNION.EDU

DOING-LAB-NOTEBOOK

SELF

Contents

Home 3

Announcements 3

Something else 3

Schedule 5

Schedule 5

Meeting Notes 7

Lab Meeting 1 9

Lab Meeting: 1 11

Lab Meeting 2 13

Lab Meeting: 2 15

Weekly Notes 17

GD Week 1 19

GD Weekly Notes: 1 21

Shared/Overall To-do 21

Phylo-learning To-do 21

Bio-xAI To-do 21

Misc Notes 23

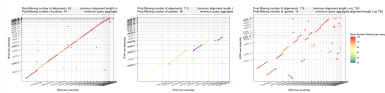
6 Jupyter Notebook Habits That Make Your Work Easy to Return To 25

6 Jupyter Notebook Habits That Make Your Work Easy to Return To 27

MT proposal 37

About 43

Home



The shared electronic lab notebook of the Doing Com-Bio Lab.

Announcements

Something

Something else

Schedule

Weekly Schedule

Schedule

check often as things may change

W	TOPIC	Reading	Due
1	What is bioinformatics: what are computers and living cells good at?	Bioinformatics for Dummies (BFD) Ch. 1 & 2	Survey, Quiz 0
2	Algorithms & the Genetic Code	Think Python (TP) Ch. 1, 2, 3.4-3.7 & Molecular & Cell Biology (MCFD) Ch. 1, 6, 7	HW 1, Quiz 1
3	Modular Programming & Proteins	TP Ch. 5.1-5.7, 5.11, 6.1 & MCFD Ch. 16, 17	HW 2, Quiz 2
4	Loops & Sequencing	TP Ch. 6.1-6.1, 7.1-7.3, 7.7 & MCFD Ch. 19	HW 3, Quiz 3
5	Sequence Processing & Disease	BFD Ch. 7	HW 4, Quiz 4
6	Sequence Alignment	BFD Ch. 9, TP Ch. 8	HW 5, Quiz 5
7	Multiple Sequence Alignment	BFD Ch. 13	Project 1 Outline

W	TOPIC	Reading	Due
8	Polymerase Chain Reaction	BDF Ch. 13, 9 cont.	Project 1
9	Phylogenetics	BFD Ch. 13 cont.	Project 2 Check-in
10	Gene Prediction	BFD Ch. 5, pp. 145-153	Project 2

Meeting Notes

Lab Meeting 1

Lab Meeting: 1

June 15, 2026

1. lab meeting structure (first few weeks)
 - 10-15 mins each for project Q&A
 - maybe walk through notebooks from the previous week, if helpful
 - flash journal club
2. daily scrum - slack
 - home or lab
 - what you'll focus on
 - to know what Qs to expect or work together
 - to avoid doing the same thing
 - to get ideas from each other
 - approx hours
 - to synchronize if helpful
 - to know when to expect Qs or As
3. working notebooks vs. notebook notebooks
 - Medium post on [good jupyter nb habits](#)
 - working colab for now - for reproducibility
 - data stored in the same google drive folder should be reproducible across accounts..
 - Union supposed to be getting a compute cluster
 - notebook notebooks can be edited in anything
 - I use VS code with googlde drive desktop synching files

- could also use colab notebooks, just different ones from working/explorations notebooks
 - could also use online editor that allows md format (eg. Text Editor google drive app)
 - don't require reading in data?
4. github seqADAGE <https://github.com/georgiadoing/seqADAGE>
 - fork
 - clone
 - make project folder within /Py/
 - we'll start code review in a couple weeks
 - [github tutorials](#)
 - maybe try the first..
 5. data in google drive folder (for now)
 - later will use Open Science Framework: <https://osf.io/>
 - maybe [this one](#) but maybe a *new* one
 - could create accounts (union email?)
 6. Misc.
 - my office key and keycard

Lab Meeting 2

Lab Meeting: 2

June 22, 2026

- testing texteditor.co

Weekly Notes

GD Week 1

GD Weekly Notes: 1

June 15, 2026

Shared/Overall To-do

- ☐ Implement tuner method in adage class
- ☐ E coli gene lists for enrichment analyses
- ☒ Get lab notebook setup
- ☐ Decide on repo structure
- ☐ Plan sampling excursion
- ☒ Key access to CAP/crochet
- ☒ Schedule lab meetings - update calendar
- ☒ Group meeting once a week
- ☐ 1-1 meetings also weekly or as-needed

Phylo-learning To-do

- ☒ Get DNA sequences into google drive
- ☐ Get P.a. DNA seq
- ☐ S aureus gene lists for enrichment analysis
- ☐ P aeruginosa gene lists for enrichment analysis

Bio-xAI To-do

- ☐ model to test SHAP tutorial on

☐ colaboration with Josh/Ara

Misc Notes

6 Jupyter Notebook Habits That Make Your Work Easy to Return To

From messy exploration to a record that still makes sense months later

6 Jupyter Notebook Habits That Make Your Work Easy to Return To

From messy exploration to a record that still makes sense months later

Published Mar 07, 2026 · Updated Apr 20, 2026 · Free: No

Here is a situation working in Jupyter Notebooks that many people will recognise.

A colleague messages you asking how a particular calculation was carried out in a study that was worked on six months ago. You know the answer is somewhere in your jupyter notebook.

You open it, confident the logic and information is in there.

Forty minutes later, you are still not sure.

The code ran. The outputs exist. But the reasoning behind the key decisions is gone. Which Clay Volume cutoff did you use and why? Was normalisation applied to the gamma ray data before or after the formation tops were picked? There is a commented-out cell that might be relevant, or might be an abandoned experiment. You can't tell.

Then you start to notice that a cell further down the page has errored as a variable is not present. After much head scratching you find out it was calculated 5 cells further down.

This is not a messy notebook problem. Messy exploration is normal and expected. This is a documentation problem: the analysis produced a result, but it did not produce a record.

In subsurface work, that distinction carries real weight. Notebooks often sit at the base of interpretations that can feed future decisions, reserve estimates, and machine learning model training pipelines. When someone asks why a decision was made, “I think that was the approach” is not an audit trail. It is a gap.

These six habits don’t slow down exploration. They ensure that what comes out of it is traceable, rerunnable, and readable by someone who was not in the room when it was written, including you, six months later.

This article does have a geoscience twist to it, but these habits can be applied to any data science project.

1. Treat the Notebook as a Decision Log, Not a Script

This is the shift that changes everything else.

Most notebooks are written as scripts: a sequence of code cells that produces an output. The problem with a script framing is that it records *what happened*, but not *why*. Code tells you that Gamma Ray values below zero were removed. It doesn’t tell you whether that was a deliberate interpretation choice, a quality control step, or a temporary workaround that was never revisited.

A notebook written as a decision log treats every significant step as something that needs to be justified, not just executed.

The practical difference is in how you use markdown cells. Not as labels for the code below them, but as the reasoning that precedes it.

Compare these two approaches to the same step.

Label (not enough):

```
## GR Cleaning
Removing negative values and normalising.
```

Decision log (what you actually need):

GR Curve: Cleaning Decisions

Negative GR values present in 3 wells (n=14 samples total). Spike pattern and depth distribution consistent with tool noise rather than formation signal. Excluded prior to any formation-level analysis.

Normalisation: min-max applied rather than z-score. Reason: preserves the shape of the GR distribution across formations, which matters for the cross-plot in section 4. Z-score would compress the tails where the lithological contrast is most visible.

The code that follows executes the decision. The markdown records it.

When you return to this notebook in four months, you don't need to reverse-engineer the reasoning from the code. It is written down. When a colleague picks it up, they know not just what was done but why, and under what assumptions.

A useful test: if you cannot write a defensible markdown cell for a step, that is a signal the decision itself needs more thought before you commit it to code.

2. Write the Header Before You Write Any Code

At the start of any notebook, you can save yourself time later on by spending two minutes writing a header cell before writing a single line of code.

```
# Project: Volve Field -- Petrophysical QC
## Notebook: 03_formation_analysis -- Curve Completeness Assessment

**Purpose:**
Assess GR and RHOB curve completeness by formation across the Volve dataset.
Output used to determine viable curve inputs for the lithology classification model.

**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.
Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

**Assumptions and exclusions:**
Intervals above casing shoe depth excluded (varies by well, see formation tops file).
GR values < 0 treated as noise and removed prior to analysis.
Brent Group subdivisions collapsed to group level due to sparse tops coverage.
```

```

**Output:**
`data/processed/curve_completeness_by_formation.csv`
`outputs/figures/completeness_heatmap.png`

**Last updated:** 2026-02-26 | **Status:** Complete

```

The two sections that matter most in geoscience or data science work are data provenance and assumptions.

Data provenance answers: where did this data come from, when was it obtained, and exactly which subset are you working with. A file path tells you nothing about whether the data came from a live database pull last week or a spreadsheet that was emailed across six months ago. Source and scope belong in the header because they are invisible everywhere else.

Additionally, a file path that resolves on your machine and nowhere else is not provenance. It is a dead end. If the data lives anywhere other than a shared location the next person is guaranteed to have access to, the header needs to explain how to get it:

```

**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.

Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

Access: Shared project drive at \\server\projects\volve_qc\data\raw\
If unavailable, contact data management or re-export from DISKOS using
the query parameters documented in `docs/data_export_notes.md`.

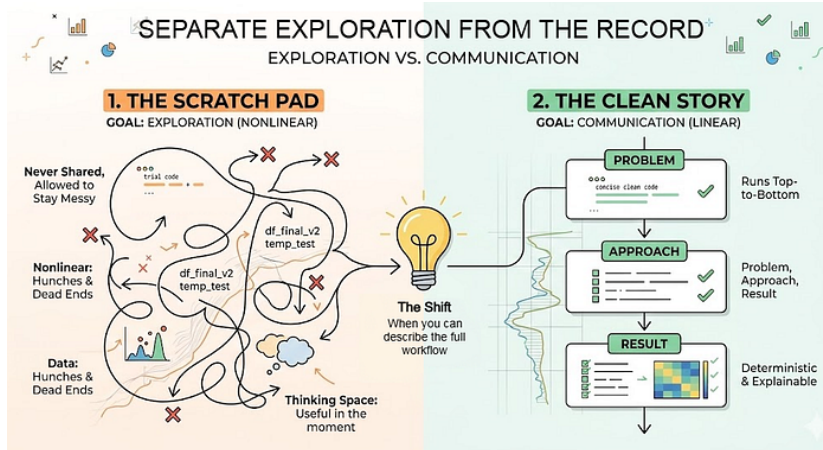
```

The fields that matter when access is not guaranteed: where the file physically lives or can be retrieved from, who to contact if access is unavailable, how it was originally obtained so it can be re-obtained the same way, and any query parameters or export settings used. A re-export with different settings is not the same dataset, and that distinction will not be obvious to the person picking up the notebook six months later.

Assumptions and exclusions are where the interpretive choices live. Collapsing Brent Group subdivisions is a meaningful geological decision. Excluding above-casing intervals is a meaningful QC decision. Both are defensible. Neither shows up in the code without a comment, and comments are the first thing to disappear during a cleanup. The header cell is permanent.

A simple status message can inform you and other notebook users whether this is a complete notebook or if it is still a work in progress.

3. *Separate Exploration From the Record*



Exploration and documentation are not the same activity, and trying to do both in one notebook produces something that serves neither purpose well.

Exploration is nonlinear by nature. You are testing approaches, hitting dead ends, trying variations you will ultimately discard. That process should be unconstrained. A notebook full of experimental cells, abandoned branches, and intermediate print statements is not a problem. It's how analysis actually happens.

The problem comes when that exploratory notebook is also treated as the record. It looks like an analysis. It produces outputs. But the path through it is not a story. It is a session log.

The habit is to maintain two notebooks per project.

The working notebook is where exploration lives. It has no cleanliness requirement. Dead ends stay in. Commented-out experiments stay in. Variable names that only made sense at 2am stay in. It is never shared and never needs to be.

The record notebook is written once you understand the shape of the answer. It tells one story: problem, approach, result. It runs cleanly from top to bottom. Every significant decision has a markdown cell. Outputs are saved to predictable locations.

A practical signal for when to make the transition: when you can describe the approach out loud, in sequence, without referring to the notebook. If you can't do that yet, you are still likely in exploration territory.

The record notebook is not just a cleaned-up version of the working notebook. It should be written from scratch, with the benefit of already knowing what the analysis is and what functions or code snippets matter from your exploration notebook. That distinction matters. Cleaning up an exploratory notebook tends to preserve its structure while hiding its confusion. Writing the record from scratch produces something that actually reads as intended and can also highlight any issues in your initial analysis.

The record notebook matters even more as AI starts to enter the analysis workflow. A model can help generate interpretations, but someone still has to verify them. And that verification depends on having a clear record of what decisions were made and why. I've written about why verification becomes the hardest problem in technical AI work over on Substack. [Link at the end.](#)

4. Name Variables for the Person Who Comes Back

Variable naming is documentation that can't be separated from the code. Unlike comments, it can't be deleted. Unlike markdown cells, it is present on every line that references the object.

The goal isn't descriptive names for their own sake. It's names that carry two pieces of information: what the object contains, and what stage of the workflow it represents.

A stage-prefix convention makes both visible at a glance:

```
raw_logs      # loaded directly from file, unmodified
clean_logs    # nulls removed, depth range trimmed, curves validated
logs_with_vcl # Vcl appended from a clay volume calculation
train_logs    # subset used for model fitting
test_logs     # held-out subset, not seen during training
```

For model objects, include the algorithm and the target variable:

```
model_rf_lithology    # random forest classifier, predicts lithology class
model_lgbm_permeability # LightGBM regressor, predicts permeability (mD)
```

The rubric is simple: a name should tell you what it is and where it sits in the workflow. If you can't construct a name that does both, the object

itself may not be clearly defined yet. That ambiguity is worth resolving before writing code that depends on it.

One habit that avoids the creeping `df2`, `df3`, `df_final` problem: name the object at creation, not at use. If you plan to rename it later, you will very unlikely rename it later.

5. The Kernel Restart Is Not Optional

Before closing any record notebook, restart the kernel and run all cells from top to bottom. If it doesn't complete cleanly, it is not finished.

The most common notebook failure mode is hidden state: a variable that was defined in an earlier session and no longer exists in the code, a file loaded from a path that no longer resolves, an import that worked once because another library happened to already be in memory. In the current session, everything appears correct. On a different machine, or six months later, it fails silently or not so silently at a cell with no obvious explanation.

The restart-and-run check is the only reliable way to surface these problems before they surface for someone else.

The practical objection is compute time. Some steps are expensive and re-running a full model training pass or reloading a large dataset on every review pass isn't realistic. The pattern that handles this cleanly is to cache intermediate outputs to `data/processed/` and load from cache in subsequent steps. The record notebook reads the cached output rather than re-executing the expensive step. The reproducibility standard still holds: the notebook runs top-to-bottom, deterministically, it just does not re-derive everything from raw data in a single pass.

The distinction worth being clear on: a notebook that produces the same outputs given the same inputs is reproducible, regardless of whether it uses cached intermediates. A notebook that only works when run in a specific order, with specific variables already in memory from a previous session, is neither reproducible nor fast. It is a future debugging problem waiting to surface at the worst possible moment.

6. Apply the Cold Open Test Before Closing

The first five habits are about writing the notebook. This one is about checking whether it actually works without you.

Before treating any record notebook as complete, try this: close it, open

it fresh, and ask whether a colleague could follow it without asking you a single question. Not a full peer review. Just a five-minute scan. Can they identify the purpose from the header? Can they locate the key decisions in the markdown? Do they know what to do with the outputs?

If the answer to any of those is no, the notebook is not finished regardless of whether it runs cleanly.

In subsurface work this test has particular weight because notebooks routinely outlive the context in which they were written. A petrophysicist moves to another asset, goes on leave, or leaves the organisation. Six months later someone needs to understand what was done and why, and the only record is the notebook. It has to stand on its own.

A practical checklist to run before marking any record notebook complete:

```
## Notebook Sign-Off Checklist
[ ] Purpose is clear from the header without opening any code cells
[ ] Every significant decision has a markdown explanation, not just code
[ ] Data source and date of extraction are recorded in the header
[ ] Assumptions and exclusions are explicitly stated
[ ] Output file names make their contents obvious without opening them
[ ] Header accurately reflects what the notebook actually does
[ ] Kernel restarted and all cells run top-to-bottom without error
```

The last point on the header is easy to overlook. A header written at the start of a project often doesn't reflect what the notebook became by the end of it. A notebook titled "exploratory curve QC" that ended up producing the formation completeness dataset used in model training is misleading to anyone who encounters it later. Update the header when the scope changes. It takes thirty seconds and it is the first thing anyone reads.

The cold open test is the difference between a notebook that is technically complete and one that is actually usable. In a field where analysis is regularly revisited, audited, or handed over, that gap matters more than it might seem.

One Habit at a Time

These habits compound, but they don't all need to start at once.

If applying all six on your next project feels like too much overhead, start with the header cell. Write it before you write any code. It takes two minutes, it forces you to state the purpose and the assumptions before you

have committed to any of them, and it is still there when you come back to the notebook in three months wondering what it was for.

The goal is not notebooks that look rigorous. It's notebooks that can be explained, re-run, and handed over without a conversation first. A header cell, honest markdown at each decision point, a clean top-to-bottom run, and sixty seconds asking whether a colleague could follow it cold gets you most of the way there without slowing down the exploration that precedes it.

These habits matter even more as AI starts to enter the analysis workflow. When a model generates an interpretation or a processed output, someone still needs to verify it. That verification depends on having a clear record: what data went in, what decisions shaped it, and what the output actually represents. A notebook without that record doesn't just make your own work harder to return to. It makes AI-assisted work harder to trust. I explored this problem directly in a recent piece on Substack — specifically why verification becomes the binding constraint in technical domains, and what that means for how you build and use AI tools.

→ [The Jagged Frontier: Why AI Amazes and Frustrates in Equal Measure](#)

MT proposal

Mukundan Thanigaivelan

Georgia Doing, PhD

Union College Undergraduate Research - Summer Research Proposal

01/29/2026

Analyzing Different Transfer Functions for Microbiological Transfer Learning

Framework:

One of the most well-studied microbes in the biological literature is *Escherichia coli*, with over 970,000 genomes available in public databases (Ussery, 2024). However, over 99% of living microbes are unknown in science (Jiao et al., 2021). With the few microbes which are known, many including *S. aureus* are less extensively studied. An approach to attempt to understand these other microbes better is transfer learning, where a machine learning model is first pre-trained on a large dataset (such as the one for *E. coli*) and then fine-tuned with one of these microbes' datasets. However, fine-tuning such a model involves a way to map features from the original, larger dataset to features in the smaller dataset. The means of this mapping is through a transfer function, of which there are many different methods.

Our research attempts to evaluate different transfer functions to align *E. coli* features to *S. aureus* features. The different methods that can be used as transfer functions include *k*-mer mapping with tools such as Salmon (Patro et al., 2015), nucleotide-based alignment with BLAST (Healy, 2007), and a higher level protein-based alignment with models such as AlphaFold (Jumper et al., 2021). Different transfer functions can have varying effects on model performance, and we aim to evaluate these functions on which ones assist in training a better model with *E. coli* providing the pre-training dataset and *S. aureus* providing the fine-tuning dataset.

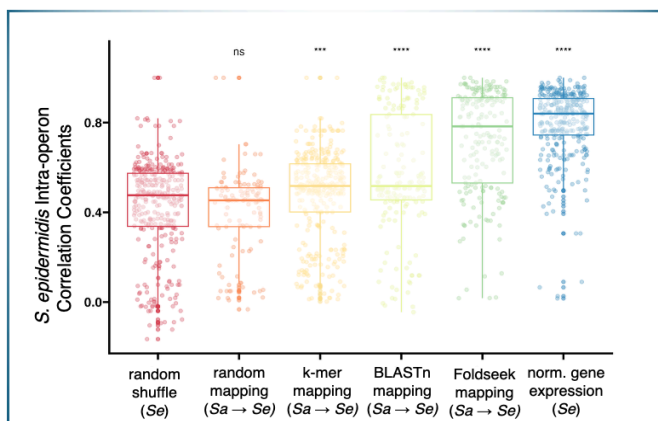


Figure 1 In this side-by-side box plot, *S. aureus* was mapped to *S. epidermidis* through different transfer learning functions. The first two box plots on the left represent negative control groups, either with random mapping or shuffling the genome. The three box-plots afterward represent *k*-mer mapping, nucleotide alignment, and protein-based mapping, from left to right. The last box plot is a positive control group, where *S. epidermidis* was mapped to its own genome. The three functions each had different effects on the model, as seen with the varying correlation coefficients on the vertical axis.

The side-by-side box plot in Figure 1 resulted from a previous analysis on different transfer functions and mapping *S. aureus* to *S. epidermidis*. In this project, we take this research as a precedent for transfer learning and build off of it to map an *E. coli* dataset to a *S. aureus* dataset. The *E. coli* dataset we will use is ten times larger than the pre-training dataset used in

this previous analysis, with over 20,000 samples total. Also, *E. coli* is more distantly related to *S. aureus* in comparison to this past research, which might reveal new insights about transfer learning performance and phylogenetic distance.

Goals:

This project aims to answer the following research **question**: which transfer function produces the best transfer learning performance when fine-tuning a machine learning model pre-trained on *E. coli* with *S. aureus* data?

In the process of answering our research question, we will learn whether different transfer functions behave differently when fine-tuning on the same dataset. We will also examine whether the phylogenetic distance between the original pre-training microbe and the lesser-studied microbe has an impact on the model performance. Model performance will be assessed using both the model's loss function and intra-operon correlations (as in Figure 1), acting as a proxy to capture biological pathways. Overall, we will also look at whether fine-tuning actually benefits the pre-trained model by increasing performance or detracts it by simply adding random noise.

We introduce the main **hypothesis** in regards to our research question: Utilizing *k*-mer mapping as the transfer function will optimize the transfer learning performance of the model, since it works better with more distantly related organisms unlike nucleotide-based BLAST alignment and predictive protein-level mapping, which might only be best for closely related organisms.

Evaluating different transfer functions for model fine-tuning is a specific part of asking a broader question: will fine-tuning actually benefit or disadvantage the model? In one part, introducing a smaller dataset could help with mapping genes and sequence alignment due to similarities in the pre-training and fine-tuning datasets. However, the new dataset could simply introduce noise into the model, weakening rather than strengthening it. This research hopes to address this higher-level question and investigate deeper into what characterizes different functions and their corresponding models.

The anticipated outcomes of this project include building different neural networks on different transfer functions and analyzing their performance, comparing them, and seeing the overall effect of the phylogenetic distance between the pre-training and fine-tuning microbes. This will build off of previous work done with mapping *S. aureus* to *S. epidermidis* with different functions (Tan et al., 2017), and analysis will be used to make conclusions and extract meaningful insights about transfer learning in microbiology.

Impact:

The field of microbiology has vast potential to grow with the study of many unknown and lesser known microorganisms, and one of the main reasons they are less studied is because so little data is available about these microbes. The more that scientists and researchers know about these microbes, the more research that can be done about applications of these microbes to solve or be better prepared for real-world problems.

We aim to see how using transfer learning, where we harness the power of a large dataset combined with a smaller but phylogenetically related dataset, can help the field understand lesser-known microbes better. If we find that this process yields statistically significant results, then future research can focus on utilizing transfer learning and neural networks to study other microbes based on an already well-studied microbe. The world of “microbial dark matter” still remains mysterious with so many unknown microbes, and we hope that our research will take a step towards a better understanding of this interesting but obscure world (Jiao et al., 2021).

Plan:

The main steps for this project can be split into four main stages: pre-training, evaluation, fine-tuning, and then a final thorough evaluation. The first stage will involve training a Denoising Autoencoder (a type of neural network) from scratch using a large *E. coli* dataset (Tan et al., 2017). To do this, we expect to use the Keras Python package (<https://pypi.org/project/keras/>) and build off of other tools already developed for *S. aureus* and *S. epidermidis* (<https://github.com/georgiadoing/seqADAGE>). The second stage, or the first evaluation, will be examining this pre-trained model as a control and assessing its performance, so that comparison can be made to the same model after fine-tuning.

The third part will be fine-tuning the trained model with transfer learning by utilizing different transfer functions discussed earlier. This will be divided into different parts with each using a different transfer function to map *E. coli* features to *S. aureus* features before fine-tuning. Then, the actual fine-tuning will happen, where the model is trained on the smaller dataset. This will produce different neural networks that were trained with different transfer functions for evaluation.

The final part of the project will be evaluation and interpretation. At this stage, we will compare the performance metrics of the pre-trained model only to the transfer learned model to find significant patterns. This evaluation stage will compare both models for *S. aureus* using AureoWiki (<https://aureowiki.med.uni-greifswald.de/>) to obtain pre-determined genes in existing databases as well as EcoCyc (<https://ecocyc.org/>) to assess if the model has learned *E. coli* pathways. From this, we will draw correlations and conclusions about each transfer function, its overall impact on a model's quality, and how it relates to the phylogenetic distance from *S. aureus* to *E. coli*. This process will involve pathway enrichment analysis.

Personal Goals:

I believe that this project supports my long-term career goals by allowing me to work extensively with machine learning and genomics. I am especially interested in using different types of machine learning methods in the scientific field to extract newer insights for real-world problems. Especially in using transfer learning in microbiology, there are many challenges since the feature spaces don't match for *E. coli* and the lesser-studied microbe. The genes are different, the gene names don't match, and genomes vary based on the organism. Microbial transfer learning presents a different real-world problem compared to traditional transfer learning, where

models trained on images of cats can be fine-tuned on dogs since the feature spaces are essentially the same—pixels with x and y coordinates.

Through this research project, I will gain experience with working with neural networks and transfer learning, evaluating different models and how their performance is shaped by input types, and how to conduct independent research and test statistical hypotheses to produce meaningful results. In terms of my professional career, I hope that this project aligns with my long-term goal of applying machine learning techniques in a future career in industry as a machine learning engineer or a data scientist. Overall, I believe that this project will help me develop creative problem-solving skills and experience with machine learning which will help me in pursuing a career in industry. I'm grateful to Professor Georgia Doing for agreeing to advise this project and the Committee on Undergraduate Research for reviewing my research proposal.

References

- Healy, M. D. (2007). Using BLAST for Performing Sequence Alignment. *Current Protocols in Human Genetics*. <https://doi.org/10.1002/0471142905.hg0608s52>
- Jiao, J.-Y., Liu, L., Hua, Z.-S., Fang, B.-Z., Zhou, E.-M., Salam, N., Hedlund, B. P., & Li, W.-J. (2021). Microbial dark matter coming to light: challenges and opportunities. *National Science Review*, 8(3). <https://doi.org/10.1093/nsr/nwaa280>
- Jumper, J., Evans, R., Pritzel, A., & Green, T. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 12. <https://www.nature.com/articles/s41586-021-03819-2>
- Patro, R., Duggal, G., & Kingsford, C. (2015). Salmon: Accurate, Versatile and Ultrafast Quantification from RNA-seq Data using Lightweight-Alignment. <https://doi.org/10.1101/021592>
- Tan, J., Doing, G., Lewis, K. A., Price, C. E., Chen, K. M., Cady, K. C., Perchuk, B., Laub, M. T., Hogan, D. A., & Greene, C. S. (2017). Unsupervised Extraction of Stable Expression Signatures from Public Compendia with an Ensemble of Neural Networks. *Cell Systems*, 5(1), 16. PubMed Central.
- Ussery, D. (2024). E. coli genomes, big data, and messy biology. *Open Access Government January 2025*, 48-49. <https://www.openaccessgovernment.org/article/e-coli-genomes-big-data-and-messy-biology/184909/>

About

References

